

**Strings y clases envoltorio**

| No. | Respuesta                          | Explicación:                                                                                                                                                                                                                                                                                                                                                                                        |
|-----|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | b. sb.append(" World");            | En String Builder existe el método append() para agregar texto al final. Las otras opciones no funcionan porque add() no existe en StringBuilder, += no se puede utilizar de esa manera y insert() necesita que indiquemos la posición donde queremos insertar el texto.                                                                                                                            |
| 2   | a. "banana".replace('a', 'e');     | El método replace() permite reemplazar todas las apariciones de un carácter por otro dentro de un String. Los otros métodos, como change(), modify() y swap(), no existen en la clase String.                                                                                                                                                                                                       |
| 3   | a. if (h1.equals(h2))              | equals() compara el contenido de los objetos String. Aunque h1 y h2 contienen "Bob", h2 fue creado utilizando new String(), por lo que es un objeto diferente. Si utilizamos ==, estaríamos comparando las referencias de los objetos y no su contenido, por eso debemos utilizar equals().                                                                                                         |
| 4   | c. StringIndexOutOfBoundsException | El string "Hello World" tiene 11 caracteres, pero los índices empiezan desde 0, por lo que van del 0 al 10. Cuando utilizamos charAt(11), estamos intentando acceder a una posición que no existe dentro del String. Por eso Java lanza una excepción StringIndexOutOfBoundsException durante la ejecución.                                                                                         |
| 5   | d. message                         | Los objetos String son inmutables. Cuando llamamos a toUpperCase(), Java genera un nuevo String con las letras mayúsculas, pero como no guardamos ese resultado en ninguna variable, el valor original sigue siendo "message".                                                                                                                                                                      |
| 6   | a. ticgameTAC                      | substring() y concat() generan nuevos objetos String, pero como no guardamos sus resultados, la variable a sigue teniendo "tic". Después utilizamos a += "game" + b.toUpperCase(), donde b.toUpperCase() devuelve "TAC". Esto imprime "ticgameTAC".                                                                                                                                                 |
| 7   | c. true true false false           | Las variables str3, str4 y str5 contienen el mismo texto "Hello World", aunque fueron creadas de diferentes maneras. Cuando utilizamos equals(), comparamos el contenido y obtenemos true en las dos primeras comparaciones. En cambio, cuando utilizamos ==, comparamos si las variables apuntan al mismo objeto, y como son objetos diferentes, obtenemos false en las últimas dos comparaciones. |
| 8   | a. false true false                | str1 se crea utilizando new String("example"), por lo que es un objeto diferente al almacenado en el String Pool. Por eso str1 == str2 devuelve false. Después, cuando utilizamos                                                                                                                                                                                                                   |

|    |                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|----|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    |                            | <p>str1.intern(), obtenemos la referencia del String Pool, que es la misma que utiliza str2, así que str2 == str3 devuelve true. Por ultimo, str1 == str3 devuelve false porque siguen siendo objetos diferentes.</p>                                                                                                                                                                                                                                                                                     |
| 9  | d. true false true         | <p>s1 y s3 contienen el mismo literal "text", así que Java reutiliza el objeto del String Pool y la comparación con == devuelve true. En cambio, s2 se crea utilizando new String(), por lo que tiene una referencia diferente y devuelve false. Por ultimo, s4 utiliza intern(), que devuelve la referencia del String Pool, por eso s1 == s4 es true.</p>                                                                                                                                               |
| 10 | a. 1 2 3 4 5 6             | <p>split("\\D") divide el texto utilizando como separador cualquier carácter que no sea un número. Por lo que obtenemos los números del 1 al 6.</p>                                                                                                                                                                                                                                                                                                                                                       |
| 11 | c. NullPointerException    | <p>La variable i es de tipo Integer y, al ser un atributo de la clase que no inicializamos, su valor por defecto es null. Cuando intentamos realizar la operación i + y, Java necesita convertir automáticamente el Integer a un int. Como i contiene null, no se puede realizar esa conversión y se produce una excepción NullPointerException.</p>                                                                                                                                                      |
| 12 | c. Hello, Heo, dlroW olleH | <p>String es inmutable, así que cuando utilizamos concat() dentro del método modify(), el valor original "Hello" no cambia. En cambio, StringBuffer y StringBuilder son mutables, por lo que append() sí modifica su contenido agregando " World". Después, en StringBuffer, utilizamos replace(2, sbf.length(), "o"), que reemplaza el texto desde la posición 2 hasta el final y deja "Heo". Por ultimo, en StringBuilder utilizamos reverse(), que invierte "Hello World" y produce "dlroW olleH".</p> |